

Rust Program Engineering Audit Checklist

End-to-end review for readable, editable, idiomatic, modular, robust, and memory-safe Rust

Project / crate	
Commit / tag	
Rust toolchain / edition / MSRV	
Target(s) + feature set(s)	
Auditor(s) / date	

Purpose: a practical audit system for Rust codebases. It combines official Rust/Cargo/rustdoc guidance with engineering heuristics for architecture, failure handling, concurrency, async, unsafe code, supply-chain risk, testing, and release readiness.

This document is a checklist and review framework, not a formal proof of soundness or security. Tooling such as Miri, Clippy, fuzzers, advisory scanners, and SemVer checkers provides evidence for particular classes of problems; none proves the absence of all defects.

How to use this audit

- 1. Establish scope before code review.** Complete Section 01 first. Mark every section as Applicable or N/A and record why. Do not silently skip difficult areas such as unsafe, FFI, async, build scripts, or feature combinations.
- 2. Record evidence, not impressions.** A check passes when the reviewer can point to code, a test, a design note, tool output, or an operational artifact. "Looks fine" is not evidence for B0/B1 items.
- 3. Treat severity as a release gate.** B0 findings are soundness/security/data-corruption blockers. B1 findings are release-blocking reliability/correctness issues unless explicitly risk-accepted. B2 findings are maintainability/performance debt that should be fixed or tracked.
- 4. Audit configurations, not only source files.** Rust feature flags, cfg targets, dependency resolution, build scripts, proc macros, release profiles, and supported toolchains can change program behavior substantially.
- 5. Re-run the master gate on the exact release commit.** For high-assurance code, archive the tool output, dependency tree, target/feature matrix, and audit sign-off with the release record.

Severity and status model

Level	Meaning	Default action	Examples
B0	Blocker	Do not merge/release until fixed or a qualified owner explicitly accepts the residual risk.	UB/soundness, exploitable auth/secret failures, memory corruption, irreversible data corruption.
B1	Release-blocking	Fix before release or document a dated exception with owner, mitigation, and follow-up.	Incorrect error/cancellation behavior, missing bounds, unsafe operational lifecycle, broken compatibility contract.
B2	Major quality	Fix in the change when practical; otherwise track as explicit engineering debt.	Readability, modularity, unnecessary coupling, missing docs, avoidable performance complexity.

Master release gate (one-page summary)

Done	Severity	Gate
[]	B0	No unexplained unsafe operations; unsafe contracts reviewed; Miri/model/fuzz evidence where applicable.
[]	B1	cargo fmt/check/clippy/test/doc clean for the supported target/feature matrix.

Done	Severity	Gate
[]	B1	Public API, errors, panics, features, and MSRV are documented and compatibility-reviewed.
[]	B1	Untrusted inputs bounded and semantically validated; arithmetic and allocation limits reviewed.
[]	B1	Concurrency/async shutdown, task ownership, cancellation, backpressure, and timeouts are tested.
[]	B0	Known dependency advisories reviewed; cargo-audit/cargo-deny policy clean or exceptions documented.
[]	B1	Release artifact built from recorded commit/toolchain/lockfile/profile and package contents reviewed.

Recommended audit order

01 Scope -> 02-03 reproducibility/architecture -> 04-08 readability/API/errors -> 09 unsafe -> 10-17 runtime/security boundaries -> 18-21 testing/features/docs/supply-chain -> 22-24 operations/performance/CI -> 25 PR delta -> 26 release gate.

Source tags in each section point to the research bibliography. Items without a source tag are synthesized engineering heuristics; they should be adapted to the threat model rather than treated as language rules.

Research basis and key conclusions

Safe Rust is not the same as a failure-proof program. The Rust Reference explicitly distinguishes undefined behavior from problems such as deadlocks, resource leaks, and arithmetic mistakes. This audit therefore separates memory soundness from higher-level correctness and operational safety.

Unsafe is a contract boundary. The Reference defines the operations that require unsafe and the responsibility to keep safe clients from triggering UB. Edition 2024 also makes `unsafe_op_in_unsafe_fn` warn by default, reinforcing explicit local unsafe blocks.

Idiomatic public API is an engineering safety mechanism. The Rust API Guidelines emphasize standard naming/conversion traits, type-driven validation, private fields, builders where appropriate, documentation of errors/panics/safety, and future-proofing.

Features multiply the program configurations you own. Cargo documents feature unification, recommends additive features, warns about default-feature behavior, and encourages a deliberate test strategy for feature combinations.

Tooling is layered evidence. Clippy finds common correctness/style/perf issues; rustdoc validates documentation/examples; Miri detects UB in executed paths; fuzzers explore input spaces; Loom explores concurrency schedules; RustSec/cargo-deny cover known dependency risks; cargo-semver-checks detects many but not all API breaks.

Suggested crate-level lint posture

```
// lib.rs or main.rs - tailor to project maturity
#![deny(unsafe_op_in_unsafe_fn)]
// Libraries may also choose:
// #![warn(missing_docs)]

// Prefer Clippy defaults + selected pedantic/restriction lints.
// Do NOT enable clippy::restriction as a whole.
```

Suggested baseline commands

```
cargo fmt --all -- --check
cargo check --workspace --all-targets
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
cargo test --doc
cargo doc --workspace --no-deps
cargo tree -d
cargo tree -e features
cargo audit
cargo deny check

# When applicable:
cargo +nightly miri test
cargo semver-checks
cargo +nightly fuzz run <target>
```

Do not blindly add `--all-features` to every command. It is appropriate when every feature combination is intended to compose; otherwise run the explicitly supported matrix. Cargo recommends additive features and warns that exhaustive feature combinations can grow exponentially.

01. Audit scope, threat model, and invariants

Do not audit syntax in isolation. Define what must never happen, which inputs are hostile, and what compatibility promises exist before judging the implementation.

Research anchors: S4, S5, S6

Done	Lvl	Audit check	Evidence / proof
[]	B1	Write a one-paragraph system purpose statement: what the crate/program does, what it explicitly does not do, and which correctness properties matter most.	README / design note
[]	B0	List protected assets and catastrophic outcomes: UB, memory corruption, privilege loss, secret disclosure, signature/auth bypass, irreversible data corruption, or consensus/state divergence.	Threat model
[]	B0	Mark every trust boundary: CLI args, config, files, network, IPC, database rows, FFI, plugins, environment variables, serialized data, user callbacks, and dependency-generated code.	Boundary map
[]	B1	Define core invariants in plain language before reviewing implementation. Each invariant should name where it is established and where it may be relied upon.	Invariant register
[]	B1	Classify the deliverable: library, CLI, daemon/service, embedded/no_std, async/networked, FFI-heavy, unsafe-heavy, or mixed. Mark non-applicable sections rather than silently skipping them.	Audit cover sheet
[]	B1	Define supported targets, operating systems, architectures, Rust edition, MSRV, feature sets, and whether nightly is permitted.	Support policy
[]	B2	Define performance/SLO constraints separately from correctness constraints so optimization cannot quietly weaken safety.	SLO / perf budget
[]	B1	Define the panic policy: where panic is acceptable, where it is a bug, and what happens if a worker/task/thread panics.	Error policy

Done	Lvl	Audit check	Evidence / proof
[]	B1	For libraries, define the SemVer/public API compatibility contract and any unstable/experimental namespaces or features.	Versioning policy
[]	B2	Record audit evidence against a specific commit SHA, toolchain, target set, feature set, and dependency lockfile.	Audit record

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

02. Repository and toolchain reproducibility

A codebase is not auditable if reviewers cannot reliably reproduce the same build inputs and tool behavior.

Research anchors: S13, S15, S16, S30

Done	Lvl	Audit check	Evidence / proof
[]	B1	Commit an explicit edition in every package manifest; avoid relying on historical defaults.	Cargo.toml
[]	B1	Declare package.rust-version when an MSRV is part of the support contract, and test it in CI.	Cargo.toml + CI
[]	B1	Use rust-toolchain.toml when the project requires a pinned channel/version or components such as rustfmt/clippy.	rust-toolchain.toml
[]	B1	For Edition 2024 workspaces, understand resolver=3 / Rust-version-aware resolution; virtual workspaces should set the resolver explicitly.	Workspace manifest
[]	B1	Keep generated source, vendored code, build artifacts, credentials, local IDE state, and platform-specific junk clearly separated and intentionally ignored or checked in.	.gitignore / repo tree
[]	B2	Make the clean-room bootstrap path documented: required system packages, native libraries, environment variables, code generation steps, and optional tooling.	CONTRIBUTING.md
[]	B1	CI builds intended release artifacts with --locked; use --frozen where network access must also be forbidden.	CI logs

Done	Lvl	Audit check	Evidence / proof
[]	B1	For applications/binaries, commit Cargo.lock. For libraries, if a lockfile is committed for CI reproducibility, also test latest-compatible dependency resolution because downstream users are not constrained by your lockfile.	Lockfile policy
[]	B2	Ensure local developer commands and CI commands are the same or share one task runner/script to avoid configuration drift.	justfile/Makefile/scripts
[]	B2	Keep repository-level configuration small and documented; surprising RUSTFLAGS, target configs, or linker overrides must be justified.	.cargo/config.toml

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

03. Workspace, crate, and module architecture

Modularity should reduce the amount of code a maintainer must understand for a change, not merely increase the number of files or crates.

Research anchors: S2, S31

Done	Lvl	Audit check	Evidence / proof
[]	B1	Each crate has a single coherent responsibility and a reason to exist as an independent compilation/API boundary.	Workspace graph
[]	B2	Avoid micro-crates that only forward symbols unless they enforce a real ownership, platform, policy, or dependency boundary.	Workspace graph
[]	B1	Dependency direction is intentional. Domain/core logic should not depend on UI, CLI, transport, storage, or framework glue unless the architecture explicitly requires it.	Dependency diagram
[]	B1	Separate pure decision/business logic from I/O where practical so the core can be tested deterministically.	Module layout + tests

Done	Lvl	Audit check	Evidence / proof
[]	B1	Keep visibility minimal: private by default, pub(super)/pub(crate) for internal boundaries, and pub only for deliberate external API.	rustdoc / source review
[]	B2	Avoid junk-drawer modules named utils/common/helpers unless they describe a narrow, stable concept.	Module tree
[]	B2	Modules are organized by domain/responsibility rather than only by data type (all structs here, all enums there) when that would scatter one change across the tree.	Module tree
[]	B2	High-level modules expose narrow facades and hide implementation modules; callers should not need internal paths to accomplish normal work.	Public paths
[]	B1	No cyclic conceptual ownership: if A and B constantly reach into each other's internals, redesign the boundary or move the shared invariant to an owning module.	Change review
[]	B2	A routine feature change should touch a small, predictable surface. If unrelated modules repeatedly change together, record and reduce the coupling.	Git history / PR sample
[]	B2	Do not introduce traits merely for 'architecture'. Use traits when there is a behavioral abstraction, multiple implementations, dynamic dispatch need, or a valuable test seam.	Trait review
[]	B2	Do not expose framework/runtime types deep into domain APIs without a reason; adapters at boundaries improve editability.	API signatures

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

04. Formatting, naming, and local readability

Readable Rust should remain understandable in diffs, compiler diagnostics, grep output, and code review without relying on IDE magic.

Research anchors: S1, S2

Done	Lvl	Audit check	Evidence / proof
[]	B1	cargo fmt --all -- --check passes with a repository-pinned style edition where consistency across editors/toolchains matters.	CI
[]	B2	Follow Rust naming conventions consistently: snake_case items, UpperCamelCase types/traits, SHOUTY_SNAKE_CASE constants/statics.	Clippy/rustfmt review
[]	B2	Names communicate domain meaning, units, ownership/lifetime expectations, and direction; avoid generic tmp/data/info/thing when a stronger name exists.	Source review
[]	B2	Conversion methods follow as_/to_/into_ conventions and iterator-producing collection methods follow iter/iter_mut/into_iter conventions.	API review
[]	B2	Functions operate at one primary level of abstraction; extract dense protocol/parsing/state-transition detail instead of mixing it with orchestration.	Function review
[]	B2	Prefer explicit intermediate variables when a long iterator/combinator chain hides domain steps or error context.	Source review
[]	B2	Comments explain why, invariants, non-obvious constraints, or safety reasoning; they do not narrate obvious syntax.	Comment review
[]	B1	Magic numbers, protocol tags, limits, timeouts, and domain constants are named and unit-qualified where ambiguity could cause defects.	Constants/config
[]	B2	Control flow makes exceptional paths visible: early return, match, if let, and helpers are used to avoid deeply nested happy-path code.	Source review
[]	B2	Dead code, commented-out implementations, stale feature gates, TODOs without ownership, and obsolete compatibility shims are removed or tracked.	Clippy + grep

Done	Lvl	Audit check	Evidence / proof
[]	B2	Public error messages and logs are actionable and identify the operation/context without requiring a backtrace to understand basic failure.	Failure tests

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

05. Idiomatic Rust and standard-trait integration

Idiomatic code reduces custom rules a maintainer must memorize and composes with the ecosystem.

Research anchors: S2, S8, S9

Done	Lvl	Audit check	Evidence / proof
[]	B1	cargo clippy runs across all relevant targets and feature sets. Treat correctness lints as defects; justify local exceptions to suspicious/style/pedantic lints.	CI / lint config
[]	B2	Do not enable clippy::restriction wholesale. Cherry-pick restrictions that encode project policy (for example unwrap_used in selected production modules).	Lint config
[]	B2	Implement common traits when semantically correct: Debug, Clone/Copy, Eq/Ord/Hash, Default, Display, Error, From/TryFrom, AsRef/AsMut, IntoIterator.	Public types
[]	B2	Use From/TryFrom and standard conversion traits instead of bespoke convert_* APIs when they express the contract accurately.	API review
[]	B2	Take the least-assumptive useful parameter type: slices over &Vec, &str over &String, IntoIterator/AsRef<Path> when the function truly needs only that capability.	Signature review
[]	B2	Avoid out-parameters; return values/tuples/structs/Results make ownership and failure clearer.	Signature review
[]	B2	Use methods when there is a clear receiver and constructors as inherent associated functions when ownership belongs to the type.	API review

Done	Lvl	Audit check	Evidence / proof
[]	B2	Use newtypes to distinguish values with the same representation but different semantics, units, trust levels, or validation state.	Type review
[]	B2	Prefer enums/custom types over boolean blindness and ambiguous Option parameters when callers need to understand a mode or policy.	API review
[]	B2	Use builders for complex construction when they improve readability, validation, or future extensibility; do not use builders for trivial two-field values.	Constructor review
[]	B2	Cloning is intentional. Review clone()/to_owned()/Arc::clone hot spots and ownership boundaries rather than treating clones as a borrow-checker escape hatch.	grep + profiler
[]	B2	Use iterator and collection APIs where they clarify intent, but do not prefer 'clever' chains over straightforward loops when the loop is easier to audit.	Source review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

06. Type-driven invariants and state modeling

Make invalid states hard or impossible to construct, and keep proof of validity attached to the value that needs it.

Research anchors: S2

Done	Lvl	Audit check	Evidence / proof
[]	B0	Every value relied upon by unsafe code, cryptographic verification, authorization, indexing, or persistent state has a clear validation point.	Invariant register
[]	B1	Constructors validate invariants or return Result; do not expose public fields that allow callers to create invalid states unless invalidity is explicitly supported.	Type API
[]	B1	Prefer validated newtypes (NonZero*, bounded IDs, parsed addresses, canonical paths, checked lengths) over repeating ad-hoc checks at use sites.	Type review

Done	Lvl	Audit check	Evidence / proof
[]	B1	State machines use enums/typstate or a similarly explicit representation instead of loosely related booleans that admit impossible combinations.	State model
[]	B1	Distinguish raw/untrusted data from parsed/validated data at the type or module boundary when confusion would be security- or correctness-relevant.	Boundary types
[]	B1	Conversions that can reject input use TryFrom/Result and preserve the original cause/context where useful.	Conversion tests
[]	B1	Numeric units are explicit (bytes vs elements, ms vs seconds, slots vs blocks, signed vs unsigned). Crossing units is centralized and tested.	Type/signature review
[]	B1	Length/index values from untrusted or wider integer domains are range-checked before use conversion, allocation, slicing, or pointer arithmetic.	Boundary tests
[]	B1	Serialized/deserialized structs are not assumed valid merely because decoding succeeded; semantic validation is separate and explicit.	Deserializer path
[]	B2	Internal invariants are asserted close to where they are established; debug_assert is used only for conditions whose absence in release cannot make unsafe behavior or security checks disappear.	Assertions review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

07. Public API design and evolution

A good public API is predictable, hard to misuse, documented, and leaves room for internal evolution.

Research anchors: S2, S3, S17, S29

Done	Lvl	Audit check	Evidence / proof
[]	B1	The exported surface is intentionally small. Run cargo doc and inspect what users actually see, including re-exports and feature-gated items.	Generated rustdoc

Done	Lvl	Audit check	Evidence / proof
[]	B1	Public structs use private fields when future invariants/evolution matter; constructors and accessors enforce the contract.	API review
[]	B2	Use #[non_exhaustive] where downstream exhaustive construction/matching would prevent planned compatible evolution.	Public enums/structs
[]	B2	Seal traits when downstream implementations would prevent safe evolution or when the trait is intended only as an internal extension point.	Trait API
[]	B2	Public dependencies are treated as part of the compatibility story when their types appear in exported signatures.	cargo tree + API
[]	B1	All fallible public functions document Errors; panicking functions document Panics; unsafe functions document Safety preconditions.	rustdoc review
[]	B2	Mark values #[must_use] when silently discarding them is likely a bug; avoid indiscriminate must_use that creates noise.	API review
[]	B2	Operator overloads are unsurprising and preserve common mathematical/domain expectations.	Trait impl review
[]	B2	Deref/DerefMut is reserved for genuine smart-pointer-like behavior; do not use Deref as inheritance or method-forwarding glue.	Trait impl review
[]	B1	Feature-gated public APIs clearly state required features and preserve compatibility expectations when features are combined.	Docs + feature tests
[]	B1	Before release, compare the public API to the previous release and run semver checking where applicable; manually review tool blind spots.	cargo-semver-checks + diff

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

08. Error handling, panic, and failure containment

Rust forces many errors into types, but a reliable program still needs a deliberate policy for which failures are recoverable, contextual, retryable, or fatal.

Research anchors: S32, S3

Done	Lvl	Audit check	Evidence / proof
[]	B1	Recoverable operational failures return Result rather than panic. Panic is reserved for violated programmer invariants or explicitly documented unrecoverable conditions.	Error policy + grep
[]	B1	Production paths that process external input do not use unwrap/expect unless the invariant making failure impossible is local, obvious, and documented.	Clippy restriction / review
[]	B1	Errors retain enough typed information for callers to distinguish classes they must handle; avoid flattening everything into opaque strings too early.	Error types
[]	B2	Add context at subsystem boundaries (which file, endpoint, key, operation) without duplicating the same message at every stack frame.	Failure output
[]	B1	Do not ignore Results from writes, flushes, joins, sends, persistence, or cleanup when failure changes correctness.	must_use/lints
[]	B1	Retry loops are bounded/backed off and only retry errors known to be transient; retries do not duplicate non-idempotent side effects.	Retry tests
[]	B1	Panics in worker threads/tasks are observed and mapped to the program's failure policy rather than silently dropping work.	Join handling
[]	B1	catch_unwind is not used as ordinary error handling. If used at plugin/FFI isolation boundaries, document which invariants remain trustworthy after a panic.	Boundary review

Done	Lvl	Audit check	Evidence / proof
[]	B1	Cleanup/Drop code does not introduce a second panic during unwinding and does not depend on fallible work that has no recovery path.	Drop impl review
[]	B2	User-facing exit codes/messages are stable enough for automation if the CLI is part of the contract.	CLI integration tests

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

09. Unsafe Rust soundness audit

Unsafe code is a proof obligation. The objective is not 'few unsafe blocks' alone, but a small, documented boundary whose safe API cannot be used to trigger undefined behavior.

Research anchors: S4, S5, S7, S19, S22

Done	Lvl	Audit check	Evidence / proof
[]	B0	Inventory every unsafe block, unsafe fn, unsafe trait/impl, extern block, unsafe attribute, raw pointer dereference, union field access, and mutable/extern static access.	rg 'unsafe extern union static mut'
[]	B0	Enable deny(unsafe_op_in_unsafe_fn) so unsafe operations inside unsafe fn still require explicit unsafe blocks.	Crate lint
[]	B0	Every unsafe block is minimal and has a SAFETY comment that states concrete preconditions/invariants, not 'this is safe'.	Source review
[]	B0	Every public unsafe fn/trait has a # Safety section describing caller obligations precisely enough to audit downstream use.	rustdoc
[]	B0	For each unsafe abstraction, prove that arbitrary safe callers cannot violate the invariants relied upon by the unsafe implementation.	Soundness note
[]	B0	Raw pointers are checked for provenance assumptions, nullness where required, alignment, initialization, dereferenceability, aliasing, lifetime, mutability, and in-bounds pointer arithmetic.	Unsafe review

Done	Lvl	Audit check	Evidence / proof
[]	B0	References created from raw pointers satisfy Rust reference rules for their full lifetime; do not create &mut aliases or references to invalid/uninitialized data.	Unsafe review
[]	B0	slice::from_raw_parts/ from_raw_parts_mut lengths are validated for element size, address range, alignment, initialization, and exclusivity.	Unsafe review + tests
[]	B0	transmute/transmute_copy is avoided where typed conversions, pointer casts, from_ne_bytes, MaybeUninit APIs, or repr guarantees can express the operation more safely.	Unsafe review
[]	B0	MaybeUninit paths prove every read is initialized exactly when required and every initialized value is dropped exactly once on success and early-return/panic paths.	Unsafe review
[]	B0	ManuallyDrop, ptr::read/write/copy, set_len, Vec::from_raw_parts, Box::from_raw, and ownership reconstruction have explicit single-owner/single-drop proofs.	Unsafe review
[]	B0	Any unsafe impl Send/Sync states why cross-thread transfer/sharing is sound, including interior mutability, raw pointers, callbacks, and external resources.	Unsafe impl note
[]	B0	repr(C), repr(transparent), repr(packed), discriminants, and layout assumptions are explicit where bytes cross ABI/storage/protocol boundaries; default Rust layout is not treated as stable.	Layout review
[]	B0	Pin/self-referential code documents pinning invariants, projection safety, drop guarantees, and which fields may move.	Pin review
[]	B0	Unsafe code has targeted tests under Miri. A clean Miri run is evidence for tested executions, not a proof of soundness.	cargo +nightly miri test

Done	Lvl	Audit check	Evidence / proof
[]	B0	Unsafe code that parses bytes, manipulates lengths, or exposes a safe wrapper is fuzzed/property-tested at the safe boundary.	Fuzz corpus/results
[]	B0	For concurrency-related unsafe/atomics, combine Miri with model testing such as Loom where applicable; one execution does not cover all interleavings.	Loom/Miri tests

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

10. Memory, ownership, lifetime, and resource management

Memory safety is only one layer. Resource leaks, retention, stack exhaustion, and lifecycle mistakes can still break production systems.

Research anchors: S6, S2

Done	Lvl	Audit check	Evidence / proof
[]	B1	Ownership matches responsibility: the component that owns a resource also owns its shutdown/cleanup contract.	Type ownership review
[]	B1	RAII guards cover files, locks, temp resources, transactions, subscriptions, and external handles when Drop can safely release them.	Resource types
[]	B1	Resources whose finalization can fail or block (flush/commit/shutdown) expose an explicit method; Drop is best-effort/non-blocking and never the only correctness path.	API review
[]	B1	Unbounded Vec/HashMap/String/queue/cache growth from untrusted or long-lived input has an explicit limit/eviction/backpressure policy.	Load/adversarial tests
[]	B1	Arc/Rc cycles, task registries, callback graphs, and caches are reviewed for logical leaks even when memory-safe.	Heap profile / ownership graph
[]	B1	Large stack allocations, recursive algorithms, and deeply nested parsing are bounded or moved to safer iterative/heap strategies when attacker-controlled depth/size is possible.	Clippy + adversarial tests

Done	Lvl	Audit check	Evidence / proof
[]	B2	Borrowed data is not kept alive accidentally by oversized owners (for example, retaining a large buffer to keep a tiny slice).	Heap profile
[]	B2	Cow, Arc, Bytes-like shared buffers, arenas, or pooling are used only when lifecycle/performance benefits are demonstrated and ownership remains clear.	Benchmark + review
[]	B1	mem::forget/leak/Box::leak are rare, deliberate, bounded, and justified as part of a process-lifetime or FFI ownership model.	grep + justification
[]	B1	Drop implementations are simple, do not acquire complex lock graphs, and are safe during partial initialization and unwinding.	Drop review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

11. Concurrency, synchronization, and atomics

Safe Rust prevents data races but not deadlocks, starvation, lost wakeups, ordering bugs, or higher-level race conditions.

Research anchors: S19, S20, S22, S24

Done	Lvl	Audit check	Evidence / proof
[]	B0	Any unsafe Send/Sync impl is reviewed as a soundness proof, not a convenience workaround.	Unsafe impl review
[]	B1	Shared mutable state has an explicit synchronization owner and invariants describing which fields must change atomically/together.	Concurrency design note
[]	B1	Lock ordering is documented for code that may hold more than one lock; tests/stress runs exercise competing acquisition paths.	Lock-order note
[]	B1	Do not perform slow I/O, callbacks into unknown code, or long computation while holding a lock unless the design explicitly requires it.	Critical-section review
[]	B1	Poisoning/recovery semantics are deliberate; do not blindly unwrap poisoned locks if partial mutation may have violated invariants.	Failure tests

Done	Lvl	Audit check	Evidence / proof
[]	B1	Channels/queues have bounded capacity or an explicit memory/backpressure justification; producer failure and receiver shutdown are handled.	Load tests
[]	B1	Thread lifecycle is owned: spawned threads have a shutdown path, join policy, panic handling, and no accidental detach when completion matters.	Shutdown tests
[]	B0	Atomic orderings are justified in comments/design notes in terms of synchronization relationships, not copied mechanically. Prefer locks when custom atomics do not provide proven value.	Atomic review
[]	B0	Lock-free structures are model-tested (for example with Loom) and reviewed for ABA/lifetime/reclamation issues as applicable.	Model tests
[]	B1	Race conditions around check-then-act, filesystem existence, caches, state transitions, and retries are tested even though Safe Rust rules out data races.	Concurrency tests
[]	B2	Contention, fairness, starvation, and priority inversion are measured under representative load when latency matters.	Load profile

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

12. Async Rust, cancellation, and task lifecycle

An async program can be memory-safe while losing messages, leaking tasks, deadlocking, or corrupting higher-level state when futures are cancelled at await points.

Research anchors: S25

Done	Lvl	Audit check	Evidence / proof
[]	B1	Every spawned task has an owner and completion policy: awaited, supervised, cancelled, or intentionally detached with bounded lifetime.	Task inventory
[]	B1	Task panics/errors are observed. Join handles are not dropped when silent failure would lose required work.	Integration tests

Done	Lvl	Audit check	Evidence / proof
[]	B1	Review every select!/race/timeout loop for cancellation safety of the losing futures; restarting a partially progressed future must not silently lose data unless documented.	Async review
[]	B1	Do not hold blocking std::sync locks across .await. For async locks, keep the guarded section minimal and understand fairness/cancellation semantics.	Clippy/review
[]	B1	Blocking filesystem, CPU-heavy, compression, crypto, and foreign calls are isolated from executor worker threads when they can stall the runtime.	Runtime traces
[]	B1	Inbound concurrency is bounded (semaphores/queues/connection limits) and backpressure reaches the producer instead of spawning unbounded work.	Load tests
[]	B1	Timeouts exist at external boundaries and include connect/read/write/overall operation semantics as appropriate; cancellation cleans up owned state.	Timeout tests
[]	B1	Graceful shutdown stops acceptance, drains or cancels in-flight work by policy, flushes required state, and has a final deadline.	Shutdown test
[]	B1	Cancellation between side effects cannot leave externally visible half-transactions without recovery/idempotency design.	Failure injection
[]	B2	Fairness assumptions in select/poll loops are explicit; hot ready branches cannot indefinitely starve shutdown/control paths.	Stress test

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

13. Parsing, serialization, protocol, and untrusted input

Treat parsing as a security boundary: decoding bytes into a Rust type does not make the data trustworthy.

Research anchors: S23

Done	Lvl	Audit check	Evidence / proof
[]	B0	All untrusted lengths/counts/depths are bounded before allocation, recursion, copy, decompression, or loop work.	Adversarial tests
[]	B0	Index/slice operations derived from input are range-checked; arithmetic uses checked_* or explicitly justified saturating/wrapping behavior before bounds decisions.	Parser review
[]	B1	Reject or define behavior for duplicate fields, unknown fields, invalid enum tags, non-canonical encodings, trailing data, malformed UTF-8, NaN/Inf, and integer width mismatches as relevant.	Parser matrix
[]	B1	Semantic validation occurs after syntactic decoding: ranges, cross-field relationships, signatures, permissions, references, and state-dependent rules.	Validation layer
[]	B1	Avoid deserialize-directly-into-privileged-state when constructors would normally enforce invariants; use raw DTO -> validate -> domain type.	Type flow
[]	B1	Compression/decompression, archive extraction, regexes, parsers, and decomposers are reviewed for amplification/algorithmic complexity and size limits.	DoS tests
[]	B1	Protocol versioning and backward/forward compatibility rules are documented and tested with golden fixtures from supported versions.	Fixture tests
[]	B1	Canonicalization occurs before equality/signature/cache-key decisions when multiple encodings can represent the same semantic value.	Canonicalization tests
[]	B1	Fuzz parsers/decoders/state transition entrypoints with structured seeds and retain minimized regression cases.	cargo fuzz / corpus

Done	Lvl	Audit check	Evidence / proof
[]	B2	Round-trip properties (encode/decode), normalization idempotence, and parse(print(x)) invariants are property-tested where meaningful.	Property tests

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

14. Numeric, time, and boundary arithmetic

Integer and time bugs are common correctness failures even when the program is fully memory-safe.

Research anchors: S5, S6

Done	Lvl	Audit check	Evidence / proof
[]	B1	Every narrowing integer conversion is deliberate. Prefer TryFrom for externally derived values; avoid as where truncation/sign change could alter correctness.	Conversion review
[]	B1	Overflow behavior is explicit at security/correctness boundaries: checked, saturating, wrapping, or overflowing operations are chosen intentionally.	Arithmetic review
[]	B1	size calculations check multiplication/addition before allocation or pointer/slice math.	Boundary tests
[]	B1	Use usize for in-memory indexing only after validating conversion from protocol/storage widths; do not let machine word size redefine a wire format.	Cross-target tests
[]	B1	Time durations carry units in types/APIs; avoid raw integer 'timeouts' without a documented unit.	API review
[]	B1	Use monotonic time for elapsed time/timeouts and wall clock for calendar timestamps; do not compare them as interchangeable notions.	Time logic tests
[]	B1	Clock rollback/forward jump, expiry boundary equality, zero/maximum durations, and deadline overflow are tested where time affects authorization or correctness.	Boundary tests

Done	Lvl	Audit check	Evidence / proof
[]	B2	Floating point comparison/serialization rules are explicit when NaN, signed zero, precision, ordering, or determinism matters.	Numeric tests

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

15. Filesystem, process, environment, and OS integration

OS interfaces add races, encoding, permissions, and trust assumptions that Rust's type system cannot remove.

Done	Lvl	Audit check	Evidence / proof
[]	B1	Paths from untrusted input are constrained to allowed roots after an appropriate canonicalization/descriptor-based strategy; test traversal and symlink cases.	Filesystem tests
[]	B1	Security-sensitive file creation sets intended permissions and avoids predictable/insecure temporary paths.	Platform tests
[]	B1	Updates that must be crash-safe use a deliberate atomic-write/fsync/rename strategy appropriate to the target filesystem and threat model.	Crash test/design note
[]	B1	TOCTOU-sensitive code does not rely on exists()/metadata checks followed by a separate privileged operation when an atomic OS primitive or open-handle check is available.	Race review
[]	B1	Environment variables/config files are treated as untrusted configuration at privilege boundaries and parsed with explicit defaults/validation.	Config tests
[]	B1	Child processes use explicit executable paths/arguments where security matters; avoid shell interpolation with untrusted text.	Process tests
[]	B1	Child stdout/stderr handling cannot deadlock on full pipes; process timeout, kill, and reap behavior is defined.	Integration tests
[]	B2	Unix/non-Unix path and OsStr encoding assumptions are isolated; avoid needless UTF-8 conversion of paths/OS strings.	Cross-platform tests

Done	Lvl	Audit check	Evidence / proof
[]	B1	Signals/service stop events integrate with graceful shutdown and do not bypass required consistency steps without policy.	Shutdown tests

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

16. FFI, ABI, native libraries, and unsafe external resources

FFI concentrates unsafety: ownership, layout, unwind, threading, callbacks, and lifetime rules must be explicit on both sides.

Research anchors: S21, S4, S5

Done	Lvl	Audit check	Evidence / proof
[]	B0	Every extern declaration is checked against the exact C/C++/system signature, calling convention, constness, width, alignment, and platform ABI.	Header/binding diff
[]	B0	repr(C)/repr(transparent) is used where layout is exchanged; default Rust enum/struct layout is never assumed stable across FFI.	Type review
[]	B0	Ownership of pointers/handles is documented: who allocates, frees, retains, borrows, and on which thread; nullability is explicit.	FFI contract
[]	B0	Callbacks cannot outlive captured Rust state; callback registration/unregistration and foreign reentrancy are tested.	Lifecycle tests
[]	B0	Panics/exceptions do not cross an ABI boundary that does not permit unwinding. Use the correct -unwind ABI only when the full cross-language unwind contract is intentional.	FFI tests
[]	B0	Foreign exceptions entering Rust across non-permitted boundaries are prevented by the integration design.	C/C++ wrapper review
[]	B0	Foreign buffers are validated before making Rust slices/references; mutable aliasing assumptions are documented.	Unsafe review
[]	B1	Native library version/runtime availability is part of deployment compatibility and checked at startup/build as appropriate.	Deployment tests

Done	Lvl	Audit check	Evidence / proof
[]	B1	Bindgen/generated bindings are reproducible or pinned, reviewed when updated, and separated from safe wrappers.	Generated diff
[]	B1	Expose a small safe Rust wrapper that centralizes unsafe calls and translates foreign error codes/resources into Rust types.	Module API

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

17. Secrets, authentication, authorization, and security-sensitive logic

Security properties should be represented as explicit checks and boundaries, not assumptions sprinkled across application code.

Done	Lvl	Audit check	Evidence / proof
[]	B0	Authorization is checked on the authoritative object/action, not inferred from UI paths, prior filters, or caller claims.	Security tests
[]	B0	Authentication/verification results are typed or otherwise difficult to confuse with unverified input; no fall-through to success on parse/verification errors.	Type flow
[]	B0	Do not invent cryptographic primitives/protocols. Use established implementations and review algorithm/mode/parameter choices against the application threat model.	Crypto design review
[]	B0	Randomness used for keys/tokens/nonces comes from a cryptographically suitable source required by the chosen library/protocol; deterministic test RNGs cannot leak into production paths.	Config + tests
[]	B0	Secrets are excluded from Debug/Display/logging/metrics/errors and crash reports. Redaction is tested.	Log tests
[]	B1	Secret lifetime/copying is minimized where realistic, while recognizing that zeroization guarantees depend on compiler/runtime/platform details and must not be overstated.	Secret handling review

Done	Lvl	Audit check	Evidence / proof
[]	B0	Comparisons that must be constant-time use a vetted constant-time primitive; ordinary == is not assumed constant-time for secret material.	Crypto review
[]	B1	Rate limits, lockouts, replay windows, nonce uniqueness, and idempotency are defined where attackers can repeat expensive or state-changing operations.	Abuse tests
[]	B1	Security-sensitive configuration fails closed or has explicit safe defaults; missing env/config cannot silently disable protections.	Startup tests
[]	B1	Audit/log events for security decisions carry identifiers and outcomes but not credentials/secrets.	Observability review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

18. Testing strategy and regression quality

Testing should target invariants and failure modes, not chase a coverage percentage without meaning.

Research anchors: S18, S11, S22, S23, S24

Done	Lvl	Audit check	Evidence / proof
[]	B1	Unit tests cover local state transitions, edge cases, and private invariants close to the implementation.	src tests
[]	B1	Integration tests exercise the public API as an external crate would and cover subsystem composition.	tests/
[]	B1	Doctests compile/run representative public examples so documentation cannot silently drift from the API.	cargo test --doc
[]	B1	Every fixed defect gets a regression test at the narrowest useful level before/with the fix.	PR review
[]	B1	Negative tests cover invalid input, permission failure, I/O failure, timeout, cancellation, partial state, malformed data, and unsupported versions.	Test matrix

Done	Lvl	Audit check	Evidence / proof
[]	B1	Property tests cover algebraic/round-trip/idempotence/order invariants where generated inputs explore more state than hand-picked examples.	proptest/quickcheck
[]	B0	Fuzz targets cover parsers, unsafe safe-wrappers, decompression/decoding, protocol framing, and complex state-machine input boundaries.	cargo fuzz
[]	B0	Miri runs targeted tests when unsafe code, tricky aliasing, custom allocation, or raw-memory manipulation exists.	Miri CI/job
[]	B1	Concurrency primitives/algorithms use deterministic model testing (for example Loom) when interleavings are part of correctness.	Model tests
[]	B1	Tests are deterministic by default: control clocks, RNG seeds, environment, temp dirs, ports, and ordering; intentionally nondeterministic stress tests are isolated.	Flake rate
[]	B1	Ignored tests include a reason and owner/condition for re-enabling; critical coverage is not permanently ignored.	Clippy + review
[]	B2	Coverage reports are used to find untested risk, not as a release-quality proxy. Review uncovered unsafe/error/security branches explicitly.	Coverage review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

19. Feature flags, cfg, portability, and no_std

Conditional compilation creates multiple programs. Audit the supported configurations, not just the developer's default build.

Research anchors: S14, S30

Done	Lvl	Audit check	Evidence / proof
[]	B1	Cargo features are additive whenever possible; enabling a feature should not disable unrelated functionality or create SemVer-breaking behavior.	Feature design

Done	Lvl	Audit check	Evidence / proof
[]	B1	Avoid mutually exclusive features; if unavoidable, reject invalid combinations with <code>compile_error</code> and document the constraint.	Feature tests
[]	B1	Test default features, <code>--no-default-features</code> , <code>--all-features</code> (when valid), and explicitly supported combinations/target-specific configurations.	CI matrix
[]	B1	Document every public feature: purpose, dependencies enabled, API surface, target constraints, stability, and default status.	Crate docs
[]	B1	Use cargo tree <code>-e features</code> / <code>-d</code> to understand feature flow and duplicate builds; <code>default-features=false</code> is not assumed sufficient if another edge re-enables defaults.	cargo tree
[]	B1	<code>cfg</code> conditions have tests or compile checks on each supported branch; dead target-specific code is not trusted because host CI never compiled it.	Cross-target CI
[]	B1	Endian, pointer width, alignment, filesystem, path encoding, and atomic-width assumptions are tested/isolated for supported targets.	Target tests
[]	B1	<code>no_std</code> crates cleanly separate <code>alloc/std</code> requirements and do not accidentally pull <code>std</code> through a feature/dependency combination.	<code>no_std</code> checks
[]	B2	Prefer runtime configuration when compile-time feature combinatorics would create untestable or ecosystem-hostile behavior.	Architecture review
[]	B1	For Edition 2024/MSRV-aware resolver behavior, verify CI against the documented MSRV and latest dependencies rather than assuming resolver choices prove compatibility.	CI matrix

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

20. Documentation and maintainability knowledge

Documentation is part of the safety boundary when callers must uphold invariants, interpret errors, or choose features correctly.

Research anchors: S3, S10, S11, S12

Done	Lvl	Audit check	Evidence / proof
[]	B1	Crate-level docs explain purpose, quick start, core concepts, feature flags, error model, safety model, and important examples.	cargo doc
[]	B1	Public items are documented. For libraries with a stable API, consider warn/deny(missing_docs) at an appropriate maturity level.	rustdoc lint
[]	B1	Examples use realistic error handling (typically ?) rather than teaching unconditional unwrap as the default.	rustdoc review
[]	B1	Public functions document Errors, Panics, and Safety sections whenever those contracts exist.	rustdoc review
[]	B1	Intra-doc links resolve; rustdoc broken link warnings are clean and links point to canonical API concepts.	cargo doc
[]	B1	Doctests run in CI; examples that are intentionally no_run/ignore explain why.	cargo test --doc
[]	B2	Internal design notes explain algorithms, invariants, lock order, protocol decisions, and rejected alternatives that are not obvious from code.	docs/ADR
[]	B2	README/CONTRIBUTING commands are copy-paste tested periodically and match the current toolchain/workspace layout.	Fresh clone test
[]	B2	Changelog/release notes identify behavior/API/MSRV/feature changes that downstream maintainers need to know.	CHANGELOG
[]	B2	Comments and docs are updated in the same PR as behavior changes; stale comments are treated as defects.	PR review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

21. Dependency and supply-chain safety

Cargo makes dependencies easy to add; every dependency also expands code executed at build time or runtime and the maintenance/security surface.

Research anchors: S26, S27, S28, S33

Done	Lvl	Audit check	Evidence / proof
[]	B0	cargo audit / RustSec advisory checks are clean or every exception has a documented, time-bounded risk acceptance and reachability rationale.	cargo audit
[]	B1	cargo deny (or equivalent policy) checks advisories, licenses, banned crates/duplicates, and allowed sources.	cargo deny check
[]	B1	Git dependencies, alternate registries, path dependencies, and source overrides are explicit and justified; production builds do not silently trust unknown sources.	cargo deny sources
[]	B1	Review build-dependencies, build.rs, proc-macros, and native bindings with higher scrutiny because they execute during the build and can expand the trusted computing base.	Dependency review
[]	B1	Direct dependencies are necessary and actively maintained enough for the project's risk level; remove unused dependencies.	cargo tree / lints
[]	B2	Duplicate versions are reviewed for binary size, compile time, incompatible types, and duplicated vulnerable code; do not mechanically ban every duplicate without context.	cargo tree -d
[]	B1	Dependency default features are reviewed rather than accepted blindly, especially for networking, TLS, native code, serialization, and runtime integrations.	Cargo.toml
[]	B1	License policy is explicit and compatible with distribution obligations; generated/vendor/native licenses are included where needed.	cargo deny licenses
[]	B1	For high-assurance projects, use cargo-vet or an equivalent human audit trail for third-party crates, including relative audits on upgrades.	cargo vet

Done	Lvl	Audit check	Evidence / proof
[]	B1	Security tooling is treated as evidence, not proof: an advisory database cannot flag unknown vulnerabilities and vetting scope/criteria must match deployment risk.	Audit policy
[]	B1	Dependency update PRs show lockfile/source diff, changelog/security relevance, and test evidence; automated updates are still reviewed.	PR template
[]	B2	Consider cargo-auditable/SBOM/provenance for shipped binaries when incident response requires reconstructing the exact dependency set.	Release artifact metadata

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

22. Observability, logging, diagnostics, and operational safety

A program that cannot explain its failures safely is hard to operate and harder to audit after an incident.

Research anchors: S2

Done	Lvl	Audit check	Evidence / proof
[]	B1	Errors and logs carry stable context: operation, component, request/job identifier, peer/resource identifier, and cause where safe.	Operational tests
[]	B0	Logs, spans, metrics labels, panic hooks, and Debug output do not expose secrets, auth tokens, private keys, sensitive payloads, or protected personal data.	Redaction tests
[]	B1	Avoid unbounded/high-cardinality metric labels derived directly from user input.	Metrics review
[]	B1	Log levels reflect actionability; expected user errors do not flood error logs and genuine invariant failures are not hidden at debug.	Load log review
[]	B1	Correlation/trace identifiers propagate through async/concurrent boundaries without relying on mutable globals.	Trace review

Done	Lvl	Audit check	Evidence / proof
[]	B1	Panic/crash handling records useful context while preserving secret-redaction rules and does not pretend the process is healthy after corrupted invariants.	Crash test
[]	B2	Health/readiness signals test dependencies/state required to serve safely; readiness is not just 'process exists'.	Deployment tests
[]	B1	Operational counters exist for dropped/retried/rejected work, queue saturation, timeout, cancellation, and parse/security failures where these affect reliability.	Dashboard review
[]	B2	Debug formatting for public/domain types is useful and non-empty, but sensitive types use redacted Debug implementations.	Debug review

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

23. Performance without sacrificing correctness

Performance review begins after semantics are correct and measured. Rust makes low-level optimization possible, not automatically justified.

Research anchors: S8

Done	Lvl	Audit check	Evidence / proof
[]	B1	Define a representative benchmark/load case and measure before optimizing; reject speculative unsafe/complexity introduced without evidence.	Benchmark baseline
[]	B2	Profile allocations, cloning, locking, syscalls, serialization, and hot loops before micro-optimizing syntax.	Profiler output
[]	B2	Benchmark release/profile settings that match production, including LTO/codegen/panic strategy only when deployment actually uses them.	Profile config
[]	B1	Performance changes preserve overflow, bounds, error, cancellation, and safety checks required by the contract.	Regression tests

Done	Lvl	Audit check	Evidence / proof
[]	B2	Avoid needless collection when streaming/iterating is clearer and bounds memory; conversely, collect when it simplifies ownership and the data is known bounded.	Hot path review
[]	B2	Reserve capacities only from trusted/bounded estimates; attacker-controlled reserve sizes are capped.	Adversarial tests
[]	B2	Use zero-copy/shared-buffer techniques only when lifetime/aliasing complexity is worth the measured gain.	Benchmark + review
[]	B2	Cold error paths remain readable; do not contort the whole codebase for branch-level gains without profile evidence.	Code review
[]	B1	Latency tests include contention/backpressure/tail behavior, not only single-thread throughput.	Load tests
[]	B1	Detect performance regressions with stable benchmarks where regressions matter operationally; do not make noisy microbenchmarks hard CI blockers without variance control.	Benchmark CI

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

24. CI quality gates

CI should make the desired engineering standard executable so the project does not depend on reviewer memory.

Research anchors: S9, S18, S26, S27

Done	Lvl	Audit check	Evidence / proof
[]	B1	Formatting gate: cargo fmt --all -- --check.	CI
[]	B1	Compilation gate: cargo check --workspace --all-targets for supported baseline features, plus explicit feature/target matrix.	CI
[]	B1	Lint gate: cargo clippy --workspace --all-targets with the intended feature matrix; -D warnings only when the project is prepared to update toolchain/lint policy deliberately.	CI

Done	Lvl	Audit check	Evidence / proof
[]	B1	Test gate: cargo test --workspace (and --all-targets/feature variants as appropriate), including doctests.	CI
[]	B1	Docs gate for libraries: cargo doc --no-deps with warnings treated according to project policy and broken links resolved.	CI
[]	B0	Dependency advisory gate: cargo audit and/or cargo deny advisories with documented exception handling.	CI
[]	B1	Supply-chain policy gate: cargo deny licenses/bans/sources or equivalent.	CI
[]	B0	Unsafe-heavy crates run Miri on a focused test set; model-test custom concurrency where applicable.	Nightly CI job
[]	B1	Parsers/security boundaries have fuzzing in scheduled/continuous jobs with persisted corpora; PR smoke fuzzing is optional based on cost.	Fuzz CI
[]	B1	Libraries with compatibility promises run MSRV CI and semver/API compatibility checks before release.	CI/release
[]	B1	At least one CI job builds release artifacts with --locked on each supported tier-1 deployment target.	Release CI
[]	B1	CI permissions/secrets are least-privilege; untrusted fork code cannot exfiltrate release credentials.	CI security settings

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

25. Pull-request and change audit

Most maintainability is won or lost one change at a time. Review the delta, not just whether the final tree compiles.

Done	Lvl	Audit check	Evidence / proof
[]	B1	The PR states the invariant/behavior being changed and the user/system consequence.	PR description

Done	Lvl	Audit check	Evidence / proof
[]	B1	The diff is narrowly scoped; unrelated refactors are separated when they make safety/correctness review harder.	Diff review
[]	B1	New public API, unsafe, dependency, feature, build-script, proc-macro, FFI, serialization, or concurrency code is explicitly called out.	PR checklist
[]	B1	Tests fail before the fix when practical and cover boundary/error cases, not only the happy path.	Test evidence
[]	B0	Every new/changed unsafe block has updated SAFETY reasoning and targeted tests; reviewers verify assumptions at the caller boundary.	Unsafe review
[]	B1	Error behavior, logs, metrics, config defaults, migrations, and shutdown behavior are considered when the change touches operational state.	Integration review
[]	B1	Feature/API/MSRV changes are checked for SemVer and downstream impact.	Compatibility review
[]	B1	Dependency changes include why the dependency is needed, feature selection, license/security/source review, and lockfile diff.	Dependency review
[]	B2	Complexity added is proportional to the requirement; simpler alternatives were considered for abstractions that introduce traits/generics/macros/unsafe.	Design note
[]	B2	Docs/examples/comments are updated in the same change; stale TODO compatibility code is not left behind without a tracking issue.	Docs diff
[]	B1	The final commit passes the same local audit command suite CI will execute.	CI green

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

26. Release and production-readiness gate

A release gate aggregates the highest-risk evidence and verifies the exact artifact/dependency set that will ship.

Research anchors: S17, S26, S27, S29

Done	Lvl	Audit check	Evidence / proof
[]	B0	No unresolved B0 findings. Any exception requires explicit security/safety owner sign-off and must not claim the code is fully sound/safe.	Audit register
[]	B1	No unresolved B1 findings without a named owner, dated risk acceptance, deployment mitigation, and follow-up issue.	Risk register
[]	B1	Clean build/test/lint/doc suite on the release commit and supported feature/target matrix.	CI release run
[]	B0	RustSec/cargo-deny advisory status is current at release time; ignored advisories are revalidated against the shipped dependency tree.	Audit output
[]	B1	Library releases pass API/SemVer review; MSRV and feature changes are documented.	Semver report
[]	B1	cargo package is reviewed (including package contents) and the published artifact excludes secrets, fixtures, private keys, local configs, and unintended generated data.	cargo package --list
[]	B1	Release artifact is built reproducibly enough for the project risk level, with the intended Cargo.lock/toolchain/profile/target recorded.	Build metadata
[]	B1	Database/schema/state migrations have forward/backward/rollback or roll-forward plans and are tested on realistic prior state.	Migration test
[]	B1	Operational rollout has health signals, rollback criteria, compatibility window, and on-call/owner awareness for risky changes.	Release plan
[]	B1	Tag/version/changelog/source commit match the actual artifact; provenance/SBOM/dependency metadata is retained where incident response requires it.	Release record

Section sign-off: Reviewer _____ Date _____ Open findings / IDs _____

Appendix A - Unsafe proof worksheet

Use one worksheet per unsafe abstraction or cluster. The goal is to turn implicit assumptions into reviewable proof obligations.

Unsafe boundary / symbol	
Safe API that reaches it	
Unsafe operations used	raw pointer deref / unsafe call / unsafe trait impl / extern / union / static / attribute / other
Caller preconditions	
Invariants established before unsafe	
Invariants required after unsafe returns	
Aliasing/exclusivity argument	
Initialization/drop argument	
Lifetime/provenance argument	
Thread-safety / Send+Sync argument	
Layout/ABI argument	
Panic/early-return cleanup argument	
Miri tests	
Fuzz/property tests	
Model/concurrency tests	
Residual assumptions / unsupported Miri behavior	

Minimum SAFETY comment pattern:

```
// SAFETY: <state the exact facts established above>.
// - ptr is non-null/aligned and points to <N> initialized T values
// - the allocation outlives the returned reference/slice
// - no mutable alias exists for that lifetime
// - len was range-checked before pointer arithmetic
unsafe { /* minimal unsafe operation */ }
```

Appendix B - End-to-end command matrix

Adapt this matrix to supported targets/features. A deep audit is strongest when the same commands are runnable locally and in CI.

Gate	Command	When	Pass condition
Format	cargo fmt --all -- --check	Every PR	No diff
Compile	cargo check --workspace --all-targets	Every PR	No errors/warnings per policy
Clippy	cargo clippy --workspace --all-targets -- -D warnings	Every PR	Clean or justified lint policy
Tests	cargo test --workspace	Every PR	Unit/integration/doctests pass
Docs	cargo doc --workspace --no-deps	Every PR / release	No broken links/warnings per policy
No-default feature build	cargo check --workspace --no-default-features	If supported	Compiles
All features	cargo check --workspace --all-features	If feature model is additive	Compiles
Feature graph	cargo tree -e features	Dependency changes	Expected feature flow
Duplicate deps	cargo tree -d	Dependency changes	Reviewed duplicates
Known advisories	cargo audit	PR + scheduled + release	No unaccepted advisories
Policy checks	cargo deny check	PR + scheduled + release	Advisories/licenses/bans/sources pass
Miri	cargo +nightly miri test	Unsafe-heavy changes / scheduled	No UB in exercised paths
Concurrency model	RUSTFLAGS="--cfg loom" cargo test --release <loom tests>	Custom atomics/concurrency	All explored schedules pass within model limits
Fuzz smoke	cargo +nightly fuzz run <target> -- -max_total_time=<N>	Parser/security changes	No crash/UB; corpus retained
SemVer	cargo semver-checks	Library release	No unexplained breaking changes; manual review remains
Package contents	cargo package --list	Release	Only intended files
Locked release build	cargo build --release --locked	Release	Exact lockfile used

Feature/target matrix worksheet

Target	Features	MSRV/latest	check	test	clippy/doc

Appendix C - High-signal Rust code smells

These are review triggers, not automatic defects. Each should make the reviewer ask what invariant or tradeoff justifies it.

Trigger	Reviewer question
unsafe without a nearby concrete SAFETY argument	Potential untracked soundness assumption.
unsafe impl Send or Sync	One incorrect marker impl can make safe callers trigger UB.
transmute / transmute_copy / from_raw_parts / set_len	High-density ownership/layout/initialization proof obligations.
unwrap/expect on data from users, files, network, IPC, or DB	Turns operational input into process/task failure.
as casts at protocol/index/allocation boundaries	May truncate/sign-change silently.
Vec::with_capacity/reserve from untrusted count	Memory exhaustion/amplification.
unbounded mpsc/channel/task spawn/cache	Memory/latency collapse under sustained load.
std::sync::MutexGuard held across .await	Executor blocking/deadlock/contention risk.
select!/timeout around a future with internal progress	Cancellation may lose partial data/state.
custom atomics/Relaxed ordering	Requires a memory-model argument; locks are often safer.
catch_unwind in normal business logic	Often hides a panic-policy/design problem.
Drop doing network/disk/blocking work	Failure/blocking has no explicit recovery path.
pub struct with all public fields	May freeze representation and bypass invariants.
bool, bool, Option<bool> parameters	Boolean blindness and impossible combinations.
huge prelude/glob re-export surface	Makes dependencies/ownership/API harder to see.
generic bounds wider than implementation needs	Reduces readability/flexibility and can overconstrain users.
trait abstraction with one impl and no boundary need	May add indirection without reducing coupling.
utils.rs continually growing	Missing domain ownership/cohesion.
feature changes that remove behavior	Cargo features are intended to be additive and can be unified.
build.rs/proc-macro/native dependency added casually	Executes trusted code during build and expands supply-chain surface.

serialization directly into privileged domain type	Decoding can bypass constructor validation.
Debug derive on a secret-bearing type	May leak secrets to logs/panics/tracing.
spawn then immediately discard JoinHandle	Silent task failure/lifecycle leak risk.
tests using sleep for synchronization	Flakiness and hidden scheduler assumptions.
comments explaining syntax rather than invariant	Noise can hide the reasoning reviewers actually need.

Research bibliography

Primary sources were selected from official Rust documentation and the current documentation/repositories of widely used Rust audit tools. Research snapshot: 27 September 2026. Tool behavior changes over time; pin versions in CI where reproducibility matters.

- S1. The Rust Style Guide - default formatting and readability principles.** <https://doc.rust-lang.org/style-guide/>
- S2. Rust API Guidelines - checklist for naming, interoperability, predictability, flexibility, type safety, dependability, documentation and future-proofing.** <https://rust-lang.github.io/api-guidelines/checklist.html>
- S3. Rust API Guidelines - Documentation.** <https://rust-lang.github.io/api-guidelines/documentation.html>
- S4. Rust Reference - Unsafety.** <https://doc.rust-lang.org/reference/unsafety.html>
- S5. Rust Reference - Behavior considered undefined.** <https://doc.rust-lang.org/reference/behavior-considered-undefined.html>
- S6. Rust Reference - Behavior not considered unsafe (deadlocks, leaks, etc.).**
<https://doc.rust-lang.org/reference/behavior-not-considered-unsafe.html>
- S7. Rust Edition Guide - unsafe_op_in_unsafe_fn in Edition 2024.**
<https://doc.rust-lang.org/edition-guide/rust-2024/unsafe-op-in-unsafe-fn.html>
- S8. Clippy Documentation - lint categories and recommended usage.** <https://doc.rust-lang.org/clippy/lints.html>
- S9. Clippy Documentation - usage and CI lint configuration.** <https://doc.rust-lang.org/clippy/usage.html>
- S10. Rustdoc Book - how to write documentation.** <https://doc.rust-lang.org/rustdoc/how-to-write-documentation.html>
- S11. Rustdoc Book - documentation tests.** <https://doc.rust-lang.org/rustdoc/documentation-tests.html>
- S12. Rustdoc Book - rustdoc lints including missing_docs and broken intra-doc links.** <https://doc.rust-lang.org/rustdoc/lints.html>
- S13. Cargo Book - Workspaces.** <https://doc.rust-lang.org/cargo/reference/workspaces.html>
- S14. Cargo Book - Features, feature unification, additive features and feature combinations.** <https://doc.rust-lang.org/cargo/reference/features.html>
- S15. Cargo Book - Dependency resolution and lockfiles.** <https://doc.rust-lang.org/cargo/reference/resolver.html>
- S16. Cargo Book - Rust version / MSRV declaration.** <https://doc.rust-lang.org/cargo/reference/rust-version.html>
- S17. Cargo Book - SemVer compatibility.** <https://doc.rust-lang.org/cargo/reference/semver.html>
- S18. Cargo Book - cargo test.** <https://doc.rust-lang.org/cargo/commands/cargo-test.html>
- S19. Rustonomicon - Send and Sync.** <https://doc.rust-lang.org/nomicon/send-and-sync.html>
- S20. Rustonomicon - Data races and race conditions.** <https://doc.rust-lang.org/nomicon/races.html>
- S21. Rustonomicon - FFI and unwind boundaries.** <https://doc.rust-lang.org/nomicon/ffi.html>
- S22. Miri - Undefined Behavior detection tool and limitations.** <https://github.com/rust-lang/miri/>
- S23. Rust Fuzz Book - cargo-fuzz.** <https://rust-fuzz.github.io/book/cargo-fuzz.html>
- S24. Loom - concurrency permutation/model testing.** <https://github.com/tokio-rs/loom>
- S25. Tokio select! documentation - cancellation safety model.**
<https://docs.rs/tokio/latest/tokio/macro.select.html>
- S26. RustSec Advisory Database / cargo-audit.** <https://rustsec.org/>
- S27. cargo-deny - advisories, bans, licenses and sources checks.**
<https://embarkstudios.github.io/cargo-deny/checks/index.html>

S28. Cargo Vet - dependency audit trail. <https://mozilla.github.io/cargo-vet/>

S29. cargo-semver-checks - automated SemVer API linting and stated limitations.

<https://github.com/obi1kenobi/cargo-semver-checks>

S30. Cargo Edition 2024 - Rust-version-aware resolver. <https://doc.rust-lang.org/edition-guide/rust-2024/cargo-resolver.html>

S31. Rust Reference - Visibility and privacy. <https://doc.rust-lang.org/reference/visibility-and-privacy.html>

S32. The Rust Programming Language - error handling with Result and panic.

<https://doc.rust-lang.org/book/ch09-00-error-handling.html>

S33. Cargo Vet - performing audits and relative audits. <https://mozilla.github.io/cargo-vet/performing-audits.html>

Important limitations

- Miri detects undefined behavior in executions it actually interprets and explicitly states that it cannot prove a library sound for arbitrary safe callers or explore every nondeterministic execution.
- Loom explores many concurrency schedules under its model, but its own documentation lists unsupported/incomplete memory-model behavior. Use it as model evidence, not a proof of all hardware executions.
- cargo-semver-checks documents areas it does not yet detect. Treat it as an automated API lint plus manual SemVer review, not a complete compatibility oracle.
- RustSec/cargo-audit and cargo-deny identify known advisories/policy violations. They cannot identify unknown vulnerabilities or prove that an advisory is reachable in your program.
- Clippy intentionally contains opinionated/pedantic/restriction lints. Correctness lints are high-signal; restriction and nursery groups should be selected deliberately rather than enabled wholesale.

Audit completion record

Release / commit	
Unresolved B0 findings	
Unresolved B1 findings / accepted risks	
Tool output archive / CI run	
Primary reviewer sign-off	
Security/safety reviewer sign-off (if required)	